BrickNation

Feature Reviewer Guide and Code Walkthrough

MERN e-commerce hardware store

Purpose: use this guide to explain, demonstrate, and defend the implemented features during a practical review. Code references are taken from the current repository and shortened to the essential logic.

Reviewer tip Start with the user journey: browse and filter products, open a product, complete an order, then review the delivered item. Switch to the admin journey for products, orders, users, charts, and reviews.

Layer	Technology	Main responsibility
Frontend	React, React Router, Redux, MUI	Pages, forms, filters, state, admin UI
Backend	Node.js, Express, Mongoose	API routes, authorization, business rules
Data and services	MongoDB, Cloudinary, Firebase, Nodemailer	Persistence, images, social login, email
Reporting	Recharts, PDFKit	Sales charts and PDF order receipts

1. How to present the architecture

The frontend sends HTTP requests through Redux thunk actions. Express routes attach authentication and role middleware before controllers execute. Controllers read or update Mongoose models, then return JSON consumed by reducers and React components.

Representative request path

```
// Source: frontend/src/actions/productActions.jsx; backend/routes/product.js
// frontend: Redux thunk -> backend endpoint
const { data } = await axios.get(
  `${import.meta.env.VITE_API}/products?${params.toString()}`
)
dispatch({ type: ALL_PRODUCTS_SUCCESS, payload: data })

// backend: route -> middleware -> controller
router.get('/products', getProducts)
```

Important files: backend/routes/product.js, backend/controllers/product.js, backend/models/product.js, frontend/src/actions/productActions.jsx, and frontend/src/reducers.

2. Product CRUD, images, and admin data table

What to demonstrate

- Admin creates and edits a product with name, price, description, category, seller, stock, and multiple images.
- Images are uploaded to Cloudinary and stored as public_id and secure URL objects in Product.images.
- The MUI DataGrid displays products with search, status chips, edit actions, selection checkboxes, and confirmation dialogs.
- Regular storefront queries exclude archived products; administrators can still see product status.

Multiple-image upload and persistence

```
// Source: backend/controllers/product.js
// backend/controllers/product.js
for (const image of images) {
  const result = await cloudinary.v2.uploader.upload(image, {
    folder: 'products', width: 150, crop: 'scale'
  })
  imagesLinks.push({
    public_id: result.public_id,
    url: result.secure_url
  })
}

const product = await Product.create({
  ...req.body, images: imagesLinks,
  category_id: category._id
})
```

Product fields used by the features

```
// Source: backend/models/product.js
// backend/models/product.js
images: [{ public_id: { type: String, required: true },
           url: { type: String, required: true } }],
isArchived: { type: Boolean, default: false },
ratings: { type: Number, default: 0 },
numOfReviews: { type: Number, default: 0 }
```

Explain the distinction	Delete is permanent and removes dependent reviews. Archive is the safer day-to-day admin action: it changes isArchived and hides the product from storefront queries.

3. Bulk actions and soft-delete/archive

ProductsList.jsx tracks selectedIds from the DataGrid. The admin confirms the action before Redux dispatches a bulk endpoint. The backend validates every ID, removes duplicates, and reports the affected count.

Validation and archive operation

```
// Source: backend/controllers/product.js
function getProductIds(req, res) {
  const { ids } = req.body || {}
  if (!Array.isArray(ids) || ids.length === 0 ||
      ids.some(id => !mongoose.isValidObjectId(id))) {
    res.status(400).json({ success: false,
      message: 'Please provide a non-empty array of valid product IDs' })
    return null
  }
  return [...new Set(ids.map(id => id.toString()))]
}

const result = await Product.updateMany(
  { _id: { $in: ids }, isArchived: { $ne: true } },
  { $set: { isArchived: true } }
)
```

Routes are protected by isAuthenticatedUser and authorizeRoles("admin"): DELETE /admin/products/bulk, PATCH /admin/products/bulk/archive, and PATCH /admin/products/bulk/unarchive.

4. Search, price/category/rating/review filters, and loading

The storefront builds URLSearchParams. APIFeatures translates those parameters into MongoDB filters, then applies sorting and pagination. The API returns both total and filtered counts so the UI can show progress and decide whether more products exist.

Filter query construction

```
// Source: frontend/src/actions/productActions.jsx
// frontend/src/actions/productActions.jsx
params.set('price[lte]', price[1].toString())
params.set('price[gte]', price[0].toString())
if (category) params.set('category', category)
if (rating) params.set('rating[gte]', rating.toString())
if (reviews) params.set('numOfReviews[gte]', reviews.toString())
```

Server-side filter translation

```
// Source: backend/utils/apiFeatures.js
// backend/utils/apiFeatures.js
if (priceGte || priceLte) {
  priceFilter.price = {}
  if (priceGte) priceFilter.price.$gte = Number(priceGte)
  if (priceLte) priceFilter.price.$lte = Number(priceLte)
}
if (queryCopy.category) {
  queryCopy.category_id = Array.isArray(queryCopy.category)
    ? { $in: queryCopy.category } : queryCopy.category
}
if (ratingGte) queryCopy.ratings = { $gte: Number(ratingGte) }
if (reviewsGte) queryCopy.numOfReviews = { $gte: Number(reviewsGte) }
```

Infinite-scroll trigger

```
// Source: frontend/src/Components/Home.jsx
// frontend/src/Components/Home.jsx
const observer = new IntersectionObserver(loadMore, {
  rootMargin: '300px'
})
```

Review answer Filtering is server-side, not only visual hiding. That keeps the result count accurate and avoids loading the entire catalog into the browser.

5. Reviews, ratings, eligibility, and moderation

Customer flow

- A user can submit or update one review per product only after an order containing that product has status

Delivered.

- The compound unique index { user_id, product_id } prevents duplicate review records.
- findOneAndUpdate with upsert makes the same endpoint support both create and update.
- After saving, an aggregation recalculates the product average and review count.

Purchase restriction and create/update behavior

```
// Source: backend/controllers/product.js
const hasPurchased = await Order.exists({
  user: req.user._id,
  orderStatus: 'Delivered',
  'orderItems.product': product._id
})
if (!hasPurchased) return res.status(403).json({
  success: false,
  message: 'You can review a product only after purchasing and receiving it'
})

await Review.findOneAndUpdate(
  { user_id: req.user._id, product_id: productId },
  { rating: numericRating, comment: filteredComment },
  { new: true, upsert: true, runValidators: true }
)
```

Bad-word filtering

Reusable content filter

```
// Source: backend/utils/contentFilter.js
// backend/utils/contentFilter.js
const { Filter } = require('bad-words')
const filter = new Filter()
const filterComment = comment =>
  filter.clean(String(comment || '').trim())
```

Admin review moderation

ProductReviews.jsx validates a product ID with Formik/Yup, loads all reviews for that product, and asks for confirmation before deleting. DELETE /reviews is admin-only; the controller recalculates the product rating and count afterward.

6. Orders, transactions, status email, and receipt PDF

Order creation uses a Mongoose session transaction so stock, order creation, and related writes can be committed together or rolled back on failure. An administrator changes the order status from the process-order page. The status update sends an HTML email with item rows and totals, plus a PDFKit receipt attachment.

Transaction pattern in controllers/order.js

```
// Source: backend/controllers/order.js
const session = await mongoose.startSession()
try {
  session.startTransaction()
  // validate stock, create order, update product stock
  await session.commitTransaction()
} catch (error) {
  await session.abortTransaction()
  throw error
} finally {
  session.endSession()
}
```

Email and PDF receipt path

```
// Source: backend/utils/orderEmail.js
const receipt = await createReceipt(order, productImages)
await sendEmail({
  email: order.user.email,
  subject,
  html: itemsHtml + totalsHtml,
  attachments: [{
    filename: 'bricknation-receipt.pdf',
    content: receipt,
    contentType: 'application/pdf'
  }]
})
```

- Receipt contents: order ID, date, status, product list, quantity x price, item subtotal, shipping, tax, and grand total.
- Order status changes are protected admin operations and are sent to the customer through the same email utility.

7. Validation and form handling

Formik controls form state and submission; Yup provides consistent client-side rules before the request is sent. The backend Mongoose validators remain the final enforcement layer.

NewProduct.jsx validation

```
// Source: frontend/src/Components/Admin/NewProduct.jsx
const validationSchema = Yup.object({
  name: Yup.string().trim().required('Product name is required'),
  price: Yup.number().min(0).required('Price is required'),
  description: Yup.string().trim().required('Description is required'),
  category: Yup.string().required('Category is required'),
  stock: Yup.number().integer().min(0).required('Stock is required')
})
```

Explain defense in depth	Frontend validation improves user feedback; backend validation protects the API when requests come from outside the browser.

8. UI design with Material UI

The project uses MUI components such as Paper, Button, Dialog, Rating, TextField, Alert, Chip, Stack, DataGrid, and responsive sx styling. Bootstrap remains for selected layout/carousel areas, while MUI supplies the admin and reviewer-facing controls.

- Admin pages updated with MUI: dashboard, products, orders, order details, users, and product reviews.
- Customer pages use MUI for cart, shipping, confirmation, payment, success, and checkout-step presentation.
- Review dialogs show validation/error states and disable submit until rating and comment are valid.

9. Charts and reporting

Dashboard reporting uses Recharts. SalesChart renders a line chart and ProductsSoldChart renders a bar chart. Date range controls are applied before chart data is displayed, allowing the reviewer to compare sales across a selected period.

Representative Recharts structure

```
// Source: frontend/src/Components/Admin/SalesChart.jsx
<LineChart data={sales}>
  <CartesianGrid strokeDasharray="3 3" />
  <XAxis dataKey="date" />
  <YAxis />
  <Tooltip />
  <Line type="monotone" dataKey="sales" stroke="#2563eb" />
</LineChart>
```

10. Authentication, roles, and account protection

- Redux stores authentication state and App.jsx loads the current user on startup.

• ProtectedRoute restricts admin routes in the frontend; backend middleware is the authoritative access control.

• Firebase Google/Facebook sign-in returns an ID token; the backend verifies it with Firebase Admin and accepts only supported providers.

• Password login tracks failed attempts and temporarily locks the account after five failures. Suspended users are rejected.

Firebase provider allow-list

```
// Source: backend/controllers/auth.js
const provider = decodedToken.firebase?.sign_in_provider
if (!['google.com', 'facebook.com'].includes(provider)) {
  return res.status(400).json({
    success: false,
    message: 'Only Google and Facebook accounts are supported'
  })
}
```

11. Render hosting and GitHub workflow

The deployed application is hosted on Render. In a typical Render setup, the GitHub repository is connected to one or more Render services: a frontend service builds the Vite app, while a backend web service runs the Express server. Render injects environment variables such as the frontend API URL and backend database/email credentials; secrets should never be committed to GitHub.

• Frontend build: run from frontend with npm install and npm run build. The generated dist directory is the static publish output.

• Backend service: run from backend with npm start, which starts the Node/Express server through nodemon in the current project configuration.

• The frontend uses import.meta.env.VITE_API, so the production value must point to the deployed backend API URL.

• After a push to the connected GitHub branch, Render can automatically build and redeploy the affected service. Build logs and runtime logs are checked in the Render dashboard.

Important repository finding	No .github/workflows directory is currently present in this repository. Therefore deployment is best explained as Render GitHub auto-deploy, not as a committed GitHub Actions workflow. If a separate workflow is added later, document its YAML path and trigger explicitly.

Deployment configuration to explain during review

```
// Source: Render service settings (not committed; configure in Render dashboard)
# Typical Render service settings
Frontend root directory: frontend
Frontend build command: npm install && npm run build
Frontend publish directory: frontend/dist

Backend root directory: backend
Backend start command: npm start

# Frontend environment variable
VITE_API=https://<deployed-backend>.onrender.com/api/v1
```

12. Redux state-management explanation

Actions dispatch request, success, and failure constants. Reducers store loading, data, and error state. Components use useSelector to render state and useDispatch to trigger asynchronous thunks. This keeps API concerns out of presentational components and makes loading/error behavior explicit.

Redux thunk lifecycle

```
// Source: frontend/src/actions/productActions.jsx
dispatch({ type: NEW_REVIEW_REQUEST })
const { data } = await axios.put('/review', reviewData, config)
dispatch({ type: NEW_REVIEW_SUCCESS, payload: data })
// catch -> NEW_REVIEW_FAIL with a user-facing message
```

13. How to explain every code snippet

Read each snippet from left to right and from the outside inward. First identify the statement keyword, then the function or object being used, then the values passed into it, and finally the result sent to the next layer. The following glossary explains the recurring words and symbols in the snippets.

Code word or symbol	Plain-language meaning
const	Declares a variable whose binding is not reassigned. The object or array inside it may still be changed.
let	Declares a variable that may be assigned a new value later.
async	Marks a function that can perform asynchronous work and return a Promise.
await	Pauses the async function until a Promise finishes, then gives the successful result back.
function / =>	Creates a function. The arrow form is a shorter function syntax commonly used in callbacks.
return	Stops the current function and sends a value back to its caller.
if / else	Runs one block only when a condition is true; else handles the alternative.
for / for...of	Repeats code. for...of visits each value in an iterable such as an image array.
try / catch / finally	Runs risky code, handles an error if it occurs, and performs cleanup afterward.
new	Creates an instance of a class or object type, such as a Mongoose ObjectId.
await axios.get/post/put/delete	Sends an HTTP request and waits for the server response. The verb describes the operation.
dispatch	Sends a Redux action to reducers so application state changes.

type	The action label Redux uses to select a reducer branch, such as REQUEST or SUCCESS.
payload	The main data carried by a Redux action.
{ }	An object block or object value. In a request body it groups named fields.
[ ]	An array/list. In a query it can hold multiple IDs or product images.
( )	Function parameters or a condition.
=>	Arrow-function separator: parameters on the left, returned expression or body on the right.
...	Spread syntax: copies fields from an object or values from an array.
?.	Optional chaining: safely reads a property when the value before it may be missing.
||	Logical OR; often supplies a fallback when the first value is empty or false.
===	Strict equality comparison; value and type must both match.
!	Logical NOT; reverses a true/false condition.
$gte / $lte / $in	MongoDB operators: greater-than-or-equal, less-than-or-equal, and match any value in a list.
req / res	Express request and response objects. req contains client input; res sends the HTTP result.
status(...).json(...)	Sets an HTTP status code and returns a JSON response body.

find / findOne / findById	Mongoose read operations that return many records, one record, or one ID match.
create / updateMany / findOneAndUpdate	Mongoose write operations for inserting, updating many records, or updating one matching record.
populate	Replaces a referenced MongoDB ID with the related document fields.
aggregate	Runs MongoDB pipeline calculations such as averages, counts, grouping, or totals.
map / filter / reduce	Array helpers: transform every item, keep matching items, or combine items into one result.
Promise.all	Runs independent asynchronous operations together and waits for all of them.
Cloudinary / PDFKit / Recharts	Project libraries for image storage, PDF creation, and charts.
Redux thunk	Allows a Redux action to run async API code before dispatching its result.

Line-by-line example: review eligibility

Snippet being explained

```
// Source: backend/controllers/product.js
const hasPurchased = await Order.exists({
  user: req.user._id,
  orderStatus: 'Delivered',
  'orderItems.product': product._id
})
if (!hasPurchased) return res.status(403).json({
  success: false,
  message: 'You can review a product only after purchasing and receiving it'
})
```

- const hasPurchased: create a named result variable.
- await Order.exists: ask MongoDB whether at least one matching order exists and wait for the answer.
- req.user._id: take the authenticated user ID attached by authentication middleware.
- orderStatus and orderItems.product: require the order to be delivered and to contain this exact product.
- if (!hasPurchased): if no qualifying order was found, stop the request.
- res.status(403).json(...): return a Forbidden response with a clear explanation.

How to explain the other snippets

- Product upload: loop through images, call Cloudinary, collect returned IDs/URLs, then pass the image array into Product.create.
- Filtering: read URL parameters, convert text to numbers, build MongoDB conditions, and apply them before pagination.
- Archive: validate IDs, use updateMany to set isArchived true, and return modifiedCount.

- Transactions: start a session, commit all writes on success, abort them on failure, and end the session in finally.
- Email/PDF: create receipt bytes, build HTML item rows, then send both representations to the customer.
- Redux: dispatch REQUEST, await axios, dispatch SUCCESS with payload, or dispatch FAIL in catch.

14. Feature-by-feature review checklist

Feature	Live demonstration	Code anchor
Product CRUD	Create, edit, view, admin list	product.js, NewProduct.jsx
Multiple photos	Upload 2+ images; inspect carousel	product.js / Cloudinary
Bulk delete/archive	Select rows; confirm; restore archive	ProductsList.jsx
Four filters	Price, category, rating, review count	apiFeatures.js
Review update	Delivered customer submits twice	createProductReview
Admin moderation	Search product ID; delete review	ProductReviews.jsx
Transaction	Place order; verify stock/order result	controllers/order.js
Status email	Admin changes status; inspect email	orderEmail.js
PDF receipt	Open attached PDF; verify totals	createReceipt

Validation	Submit blank/invalid form	Formik + Yup
Charts	Change date range; inspect line/bar chart	SalesChart.jsx
Social auth	Google/Facebook login	firebaseAuth.jsx
Redux	Observe request/success/error states	actions + reducers

15. Short oral-defense answers

Why use archive instead of immediately deleting?

Archive preserves recoverability and prevents an accidentally removed catalog item from appearing in the storefront. Permanent deletion remains available for deliberate cleanup and also removes dependent reviews/blueprint references.

How is a review restricted to real customers?

The backend checks the authenticated user, the exact product, and an order with Delivered status before allowing an upsert. The browser button is only a convenience; the API check enforces the rule.

Where is the grand total represented?

The order stores item, shipping, tax, and total prices. orderEmail.js renders those values in the HTML email and creates the same order summary in the PDFKit receipt.

What happens when an API call fails?

Redux dispatches a failure action with the server message, and the relevant component displays an Alert or toast. Admin bulk actions also refresh the list after success.

16. Source map

Area	Primary files
Products and filters	backend/controllers/product.js; backend/utils/apiFeatures.js; backend/models/product.js
Reviews	backend/models/review.js; backend/utils/contentFilter.js; frontend/src/Components/Product/ProductDetails.jsx

Admin review UI	frontend/src/Components/Admin/ProductReviews.jsx
Orders and email	backend/controllers/order.js; backend/utils/orderEmail.js; backend/models/order.js
Frontend state	frontend/src/actions/productActions.jsx; frontend/src/reducers; frontend/src/main.jsx
Forms and UI	frontend/src/Components/Admin/NewProduct.jsx; frontend/src/Components/Admin/ProductsList.jsx
Auth and routes	backend/controllers/auth.js; backend/routes/product.js; frontend/src/App.jsx

Directory map for source files

- backend/controllers/ - request handlers and business rules, including product.js, order.js, and auth.js.
- backend/models/ - Mongoose schemas for Product, Review, Order, User, Category, and Blueprint.
- backend/routes/ - Express route definitions and authentication/role middleware wiring.
- backend/utils/ - reusable API filtering, profanity filtering, email, password-reset, and receipt-generation helpers.
- frontend/src/Components/ - React pages and UI components, grouped into Admin, Cart, Product, Review, User, Order, and Layout folders.
- frontend/src/actions/ - Redux thunk API calls; frontend/src/reducers/ - request/success/failure state transitions.
- frontend/src/constants/ - Redux action type constants; frontend/src/Utils/ - shared frontend helpers.
- frontend/public/ - static assets such as icons and the favicon; frontend/src/main.jsx and App.jsx - application bootstrap and route map.
- tools/generate-reviewer-guide.js - reproducible PDF guide generator; BrickNation-Feature-Reviewer-Guide.pdf - generated reviewer artifact.

Final reminder Do not only describe the screen. Explain the complete path: UI event -> Redux action -> protected route -> controller/model rule -> response -> reducer -> visible feedback.